中南林业科技大学

编译原理实验报告

课程名称：	编译原理
专业班级：	软工 3 班
姓　　名：	李佳鹏
学　　号：	20245428

完成日期：2026 年 10 月

目录

实验一 词法分析.....1
一、实验目的.....1
二、实验内容.....1
三、实验理论依据.....9
四、测试数据及运行结果展示.....13

实验一 词法分析

一、 实验目的

编制一个读单词过程，从输入的源程序中识别出各个具有独立意义的单词，即基本保留字、标识符、常数、运算符、分隔符五大类，并依次输出各个单词的内部编码及单词符号自身值。

通过本次实验，掌握词法分析器的设计方法：先写出语言的词法规则，把词法规则转换为状态转换图，再把各转换图的初态连接在一起构成识别该语言的自动机，最后据此设计扫描器。扫描器作为语法分析的一个过程，当语法分析需要一个单词时被调用，它从初态出发，识别一个单词后进入终态，送出一个二元式。

二、 实验内容

（1） 确定单词的分类及内部编码，明确五大类单词各自包含哪些符号。

```
// 单词内部编码约定（五大类）
enum TokenCode {
    TK_EOF       = 0,   // 结束符（内部标志）
    TK_KEYWORD   = 1,   // 1 —— 基本保留字 (keyword)
    TK_IDENT     = 2,   // 2 —— 标识符 (identifier)
    TK_CONST     = 3,   // 3 —— 常数 (constant：整型 / 实型 / 字符串 / 字符)
    TK_OPERATOR  = 4,   // 4 —— 运算符 (operator)
    TK_DELIMITER = 5    // 5 —— 分隔符 (delimiter)
};

const std::unordered_set<std::string>& keywordTable() {
    static const std::unordered_set<std::string> kTable = {
        "auto", "break", "case", "char", "const", "continue", "default", "do",
        "double", "else", "enum", "extern", "float", "for", "goto", "if",
        "int", "long", "register", "return", "short", "signed", "sizeof",
        "static", "struct", "switch", "typedef", "union", "unsigned", "void",
        "volatile",  "while",  "inline",  "restrict",  "_Bool",  "_Complex",
"_Imaginary"
    };
    return kTable;
}
```

```
inline bool isAlpha(char c)     { return (c >= 'a' && c <= 'z') || (c >= 'A' && c
<= 'Z'); }
inline bool isIdentHead(char c) { return isAlpha(c) || c == '_'; }
inline bool isIdentBody(char c) { return isAlpha(c) || isDigit(c) || c == '_'; }

// 字符/字符串常量由引号引导，数字常量由 scanNumber 处理
if (c == '"' || c == '\'') return emit(scanQuoted(c));
if (isDigit(c) || (c == '.' && isDigit(peek(1)))) return emit(scanNumber(false));

inline bool isSingleOperator(char c) {
    static const std::string kOps = "+-*/%=<>!&|^~?#";
    return kOps.find(c) != std::string::npos;
}
// 多字符运算符通过 multiOperatorTable() 维护

inline bool isDelimiter(char c) {
    static const std::string kDelims = "(){}[],;:.";
    return kDelims.find(c) != std::string::npos;
}
```

（2） 写出 C 语言子集的词法规则（正规式），包括标识符、常数、运算符、分隔符、注释与空白。

```
// 跳过空白符（空格、制表符、回车、换行）
void skipBlank() {
    while (pos_ < src_.size()) {
        char c = src_[pos_];
        if (c == ' ')       { advance(); }
        else if (c == '\t') { advance(); }
        else if (c == '\r') { advance(); }
        else if (c == '\n') { advance(); lineStart_ = true; }
        else break;
    }
}

// 行注释：// ... 直到行尾
void skipLineComment() {
    while (pos_ < src_.size() && src_[pos_] != '\n') advance();
}

// 块注释：/* ... */，允许跨行
void skipBlockComment() {
    int l = line_, co = col_;
    advance(); advance();                   // 吃掉 "/*"
```

```
    while (pos_ < src_.size()) {
        if (src_[pos_] == '*' && peek(1) == '/') { advance(); advance(); return; }
        advance();
    }
    error(l, co, "块注释未闭合（缺少 \"*/\"）");
}

inline bool isAlpha(char c)     { return (c >= 'a' && c <= 'z') || (c >= 'A' && c
<= 'Z'); }
inline bool isIdentHead(char c) { return isAlpha(c) || c == '_'; }
inline bool isIdentBody(char c) { return isAlpha(c) || isDigit(c) || c == '_'; }

// 标识符 / 保留字扫描
Token scanIdentifier() {
    int l = line_, co = col_;
    std::string s;
    while (!atEnd() && isIdentBody(cur())) s += advance();

    // 查保留字表，决定编码是保留字(1)还是标识符(2)
    int code = keywordTable().count(s) ? TK_KEYWORD : TK_IDENT;
    return Token{code, s, l, co};
}

// 字符串常量 "..." / 字符常量 '...'
Token scanQuoted(char quote) {
    int l = line_, co = col_;
    std::string s;
    s += advance();                          // 左引号
    while (!atEnd() && cur() != quote) {
        if (cur() == '\n') break;            // 不允许跨行
        if (cur() == '\\') {                 // 转义字符处理
            s += advance();
            if (!atEnd()) s += advance();
            continue;
        }
        s += advance();
    }
    if (cur() == quote) s += advance();      // 右引号
    else error(l, co, std::string(1, quote) == "\"" ? "字符串常量未闭合" : "字符
常量未闭合");
    return Token{TK_CONST, s, l, co};        // 归入常数(3)
}

// 数字常量（整型、实型、十六进制、指数及符号处理）
```

```
Token scanNumber(bool withSign) {
    int l = line_, co = col_;
    std::string s;
    if (withSign) s += advance();            // 吞掉 '+' / '-'

    if (cur() == '0' && (peek(1) == 'x' || peek(1) == 'X')) {
        // 十六进制
        s += advance(); s += advance();
        while (!atEnd() && isHexDigit(cur())) s += advance();
    } else {
        // 十进制整数部分
        while (!atEnd() && isDigit(cur())) s += advance();
        // 小数部分
        if (cur() == '.' && (isDigit(peek(1)) || !isIdentHead(peek(1)))) {
            s += advance();
            while (!atEnd() && isDigit(cur())) s += advance();
        }
        // 指数部分
        if (cur() == 'e' || cur() == 'E') {
            size_t savePos = pos_;
            int saveLine = line_, saveCol = col_;
            std::string tmp = s;
            tmp += advance();
            if (cur() == '+' || cur() == '-') tmp += advance();
            if (isDigit(cur())) {
                while (!atEnd() && isDigit(cur())) tmp += advance();
                s = tmp;
            } else {
                pos_ = savePos; line_ = saveLine; col_ = saveCol; // 回退
            }
        }
    }
    if (cur() =='f' || cur() =='F' || cur() =='l' || cur() =='L') s += advance();
// 类型后缀
    return Token{TK_CONST, s, l, co};
}

// 在 next() 主循环中的运算符与分隔符匹配逻辑：
// 7) 运算符 / 分隔符：最长匹配（先试三字符、再试双字符、最后单字符）
if (pos_ + 2 < src_.size()) {
    std::string three = src_.substr(pos_, 3);
    if (multiOperatorTable().count(three)) {
        int l = line_, co = col_;
        advance(); advance(); advance();
```

```
        return emit(Token{TK_OPERATOR, three, l, co});
    }
}
if (pos_ + 1 < src_.size()) {
    std::string two = src_.substr(pos_, 2);
    if (multiOperatorTable().count(two)) {
        int l = line_, co = col_;
        advance(); advance();
        return emit(Token{TK_OPERATOR, two, l, co});
    }
}
if (isSingleOperator(c)) {
    int l = line_, co = col_;
    advance();
    return emit(Token{TK_OPERATOR, std::string(1, c), l, co});
}
if (isDelimiter(c)) {
    int l = line_, co = col_;
    advance();
    return emit(Token{TK_DELIMITER, std::string(1, c), l, co});
}
```

（3）绘制各类单词识别的状态转换图，合并初态构成识别该语言全部单词的自动机。

```
Token next() {
    for (;;) {
        skipBlank();                          // 跳过空白符
        if (atEnd()) return Token();          // 终态：文件结束

        char c = cur();

        // 1) 跳过注释（不构成单词）
        if (c == '/' && peek(1) == '/') { skipLineComment();  continue; }
        if (c == '/' && peek(1) == '*') { skipBlockComment(); continue; }

        // 2) 预处理指令跳过
        if (c == '#' && lineStart_) { skipped_.push_back(skipPreprocessor());
continue; }

        // 3) 转向：标识符 / 保留字状态图
        if (isIdentHead(c)) return emit(scanIdentifier());

        // 4) 转向：字符串 / 字符常量状态图
        if (c == '"'  || c == '\'') return emit(scanQuoted(c));
```

```
        // 5) 转向：无符号数状态图
        if (isDigit(c) || (c == '.' && isDigit(peek(1))))
            return emit(scanNumber(false));

        // 6) 转向：带符号数状态图（依赖上下文 operandExpected_）
        if ((c == '-' || c == '+') && operandExpected_ &&
            (isDigit(peek(1)) || (peek(1) == '.' && isDigit(peek(2)))))
            return emit(scanNumber(true));

        // 7) 转向：运算符 / 分隔符状态图（最长匹配：3 字符 -> 2 字符 -> 单字符）
        if (pos_ + 2 < src_.size()) {
            std::string three = src_.substr(pos_, 3);
            if (multiOperatorTable().count(three)) {
                int l = line_, co = col_;
                advance(); advance(); advance();
                return emit(Token{TK_OPERATOR, three, l, co});
            }
        }
        if (pos_ + 1 < src_.size()) {
            std::string two = src_.substr(pos_, 2);
            if (multiOperatorTable().count(two)) {
                int l = line_, co = col_;
                advance(); advance();
                return emit(Token{TK_OPERATOR, two, l, co});
            }
        }
        if (isSingleOperator(c)) {
            int l = line_, co = col_;
            advance();
            return emit(Token{TK_OPERATOR, std::string(1, c), l, co});
        }
        if (isDelimiter(c)) {
            int l = line_, co = col_;
            advance();
            return emit(Token{TK_DELIMITER, std::string(1, c), l, co});
        }

        // 8) 应急恢复：无法识别的字符报错
        {
            int l = line_, co = col_;
            std::string bad(1, advance());
            error(l, co, "无法识别的字符 '" + bad + "'");
        }
```

```
    }
}
```

（4）用 C++ 实现扫描器程序，采用“最长匹配＋超前搜索一个字符”的策略，支持注释与预处理指令的跳过，并对非法输入给出错误定位。

```
// 超前搜索核心函数
char peek(size_t n) const {
    return pos_ + n < src_.size() ? src_[pos_ + n] : '\0';
}

// 在 next() 中实现运算符的“最长匹配”
if (pos_ + 2 < src_.size()) {
    std::string three = src_.substr(pos_, 3);
    if (multiOperatorTable().count(three)) { // 优先尝试 3 字符运算符 (如 <<=, >>=)
        int l = line_, co = col_;
        advance(); advance(); advance();
        return emit(Token{TK_OPERATOR, three, l, co});
    }
}
if (pos_ + 1 < src_.size()) {
    std::string two = src_.substr(pos_, 2);
    if (multiOperatorTable().count(two)) {// 其次尝试2字符运算符 (如 ==, <=, ++)
        int l = line_, co = col_;
        advance(); advance();
        return emit(Token{TK_OPERATOR, two, l, co});
    }
}
if (isSingleOperator(c)) { // 最后尝试单字符运算符
    int l = line_, co = col_;
    advance();
    return emit(Token{TK_OPERATOR, std::string(1, c), l, co});
}
// 1) 跳过注释（行注释与块注释）
if (c == '/' && peek(1) == '/') { skipLineComment();  continue; }
if (c == '/' && peek(1) == '*') { skipBlockComment(); continue; }

// 2) 跳过行首的预处理指令 (#...)
if (c == '#' && lineStart_) { skipped_.push_back(skipPreprocessor()); continue; }

// 具体跳过实现：
void skipLineComment() {
    while (pos_ < src_.size() && src_[pos_] != '\n') advance();
}
```

```
void skipBlockComment() {
    int l = line_, co = col_;
    advance(); advance(); // 吃掉 "/*"
    while (pos_ < src_.size()) {
        if (src_[pos_] == '*' && peek(1) == '/') { advance(); advance(); return; }
        advance();
    }
    error(l, co, "块注释未闭合（缺少 \"*/\"）");
}

std::string skipPreprocessor() {
    std::string raw;
    while (pos_ < src_.size() && src_[pos_] != '\n') {
        if (src_[pos_] == '\\' && peek(1) == '\n') { // 支持 \ 续行
            raw += advance(); raw += advance();
            continue;
        }
        raw += advance();
    }
    return raw;
}

// 字符推进与行列跟踪
char advance() {
    char c = src_[pos_++];
    if (c == '\n') { ++line_; col_ = 1; }
    else if (((unsigned char)c & 0xC0) != 0x80) ++col_; // UTF-8 兼容处理
    return c;
}

// 错误记录函数
void error(int line, int col, const std::string& msg) {
    std::ostringstream oss;
    oss << "第 " << line << " 行第 " << col << " 列：" << msg;
    errors_.push_back(oss.str());
}

// 在无法识别非法字符时触发定位
{
    int l = line_, co = col_;
    std::string bad(1, advance());
    error(l, co, "无法识别的字符 '" + bad + "'");
}
```

（5）用实验指导书给出的示例程序与测试数据运行程序，验证输出与预期结果一致。

实验环境：Windows 11 操作系统，g++（MinGW-w64）8.1.0 编译器，C++17 标准；源程序为单文件 lexer.cpp。

三、 实验理论依据

1．词法分析的任务

词法分析是编译过程的第一个阶段（扫描阶段）。它的输入是源程序的字符流，输出是单词符号（token）序列。每个单词用一个二元式表示：（单词种别，单词符号的属性值）。其中单词种别即该单词的内部编码，通常用整数表示；属性值即单词符号自身值，如标识符的名字、常数的数值、运算符的字符串等。本程序输出的二元式形如 (1,"int")、(3,"-5")。

2．单词的分类与内部编码

按照实验要求，把所有单词分成五大类，并分别赋予 1～5 的内部编码，具体见表 1。

表 1　单词的分类与内部编码

编码	单词类别	说　明	示　例
1	基本保留字	语言中有固定含义的关键字	int　float　if　else
2	标识符	用户定义的名字	main　a　y1　scanf
3	常数	整型、实型、字符串、字符	4　-5　3.1416　"%d"
4	运算符	算术、关系、逻辑、赋值等	=　+　>=　&&　++　<<=
5	分隔符	起分隔与定界作用	(　)　{　}　,　;　:

3．词法规则

记 letter 为任一字母，digit 为任一十进制数字，hexdigit 为任一十六进制数字，则本语言各单词的词法规则可用正规式描述，见表 2。

表 2　各单词的词法规则

单词类别	正规式（或说明）
标识符	letter (letter | digit | _)*
保留字	保留字表中的某一项，共 37 个
无符号整数	digit digit*　　十六进制：0 (x | X) hexdigit+
实型数	digit* . digit+ ((e | E) (+ | -)? digit+)?
字符串常数	″(除引号与换行的字符 | 转义序列)* ″
字符常数	’(除引号与换行的字符 | 转义序列)* ’
运算符	+ - * / % = < > ! & | ^ ~ ? 及其双字符、三字符组合，按最长匹配
分隔符	( ) { } [ ] , ; : .
注释	// 至行末；/* 任意字符序列 */　　——扫描时跳过
空白与预处理	空格、制表、回车、换行；行首 # 至行尾　——扫描时跳过

4．状态转换图

按照实验步骤，先分别画出各类单词识别的状态转换图，再把各转换图的初态连接在一起，得到识别该语言全部单词的自动机。合并后的主控状态转换图如图 1 所示。

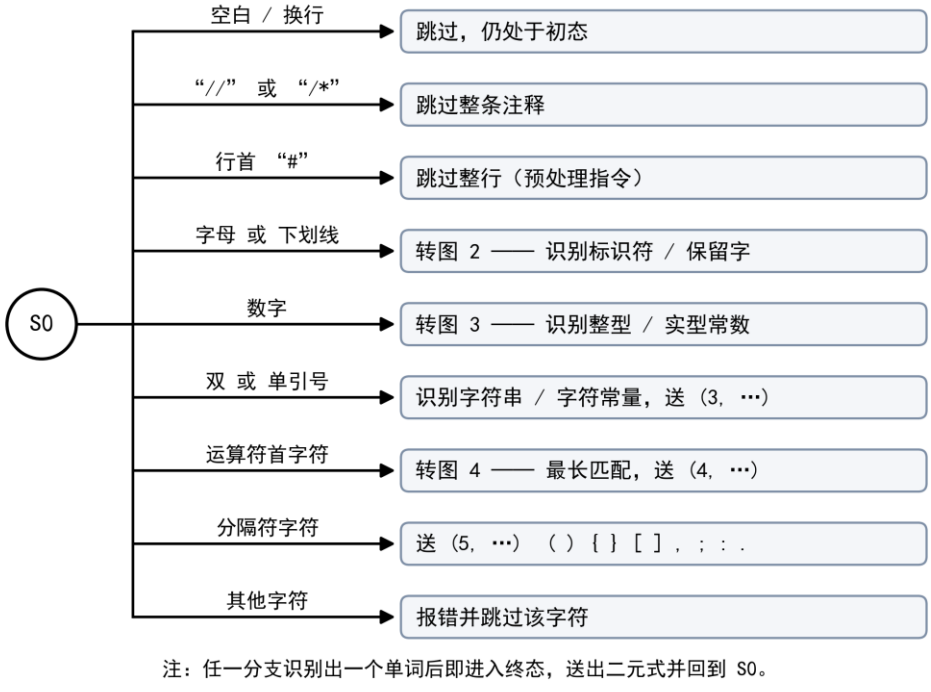


图 1　合并初态后的主控状态转换图

图 1 中，状态 S0 是一切单词识别的初态，程序每识别出一个单词、送出一个二元式后都返回 S0；遇到空白、注释和预处理指令时直接跳过，不产生单

词。各类单词的具体识别过程如图 2～图 4 所示。

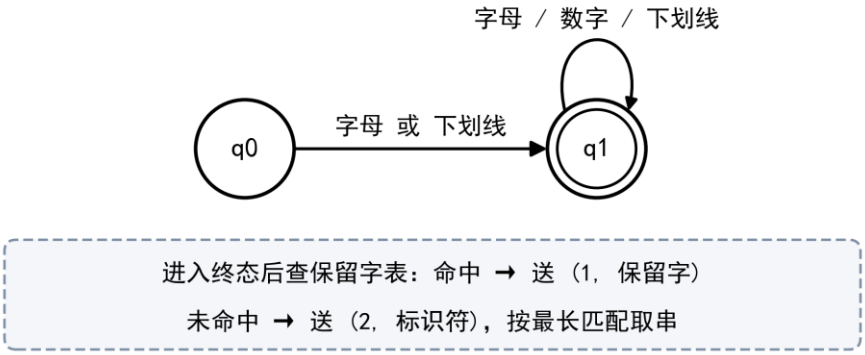


图 2　标识符与保留字的识别

图 2 中，q0 是标识符识别的初态，读入一个字母或下划线后转入终态 q1；q1 上方的自环表示可以继续读入字母、数字或下划线，由此实现最长匹配。由于标识符与保留字的识别路径完全重合，到达终态后再用已读入的字符串查保留字表，命中者为保留字，否则为标识符，二者共用同一条识别路径。

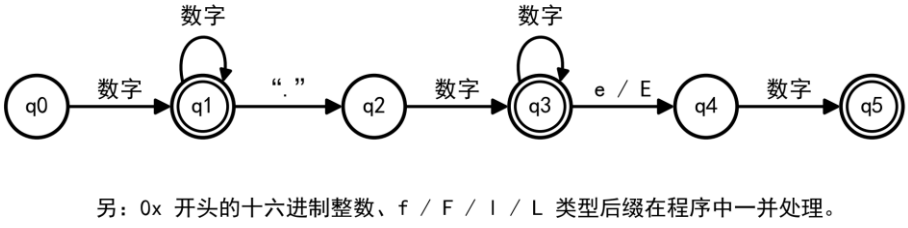


图 3　常数的识别

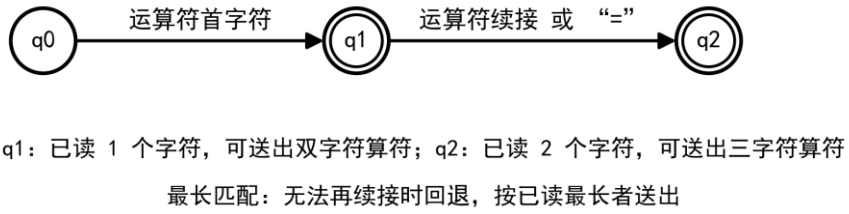


图 4　运算符的识别（最长匹配）

5．由状态转换图到确定的有限自动机

上述各状态转换图中，任一状态下对同一个输入字符最多只有一条转移，即状态转移互不冲突，因此由状态转换图直接得到的就是一个确定的有限自动机（DFA），无需再进行子集构造（确定化）和最小化处理。程序中用一重循环加分

支判断实现该自动机：读入一个字符后，按当前所处的识别路径决定后继状态，路径结束即到达终态，此时回送二元式。

为避免把一个单词的前缀误当作整个单词（例如把 >= 拆成 > 和 =、把 3.14 拆成 3 和 .14），扫描器采用最长匹配原则：到达终态后并不立即送出，而是再向前看一个字符，若该字符仍可续接则继续转移，直到不能再续接为止，然后回退这个“多看”的字符并送出当前已识别的最长单词。这就是词法分析中的超前搜索。

6．保留字与标识符的同构处理

从正规式看，所有保留字都满足标识符的构成规则，即保留字集合是标识符集合的子集。因此二者可以共用同一条识别路径：先按标识符识别出一个字符串，再查保留字表，命中则为保留字（编码 1），否则为标识符（编码 2）。程序中用哈希表（unordered_set）存放 37 个保留字，平均查找时间为 O(1)。这样既省去了为保留字单独设计状态转换图，也避免了保留字被误识别为标识符。

7．正负号的归属

字符“-”在源程序中有两种含义：一是双目算术减号，属于运算符；二是常量的符号位。若不加区分，int a=-5 中的 -5 就会被拆成 (4,"-") 和 (3,"5")，与要求输出不符。为此程序设置了一个状态变量 operandExpected，记录“下一个位置是否期待一个运算对象”：在初态、运算符之后，以及 ([{ , ; : 等分隔符之后置为真；在标识符、常数、)] 之后置为假。只有当 operandExpected 为真且正负号后紧跟数字时，才把符号并入常数。于是 a=-5 输出 (3,"-5")，而 a-b 输出 (4,"-")。

8．注释、预处理指令与错误处理

注释不是单词，扫描时直接跳过；块注释允许跨行，并检查是否以 */ 闭合。行首的 # 指令（如 #include "math.h"）不属于五大类单词，若逐个字符识别会把 math 和 h 误当作标识符，故整行跳过并单独提示。遇到无法识别的字符、未闭合的注释或字符串、非法常数（如 123abc）时，程序给出其行号、列号，跳过该字符后继续扫描，使一次运行能够报出全部词法错误。

四、 测试数据及运行结果展示

（一）测试数据一

取自实验指导书给出的示例程序，源程序如下：

```
main()
{
  int  a=-5,b=4,j;
  if(a>=b)
  j=a-b;
  else  j=b-a;
}
```

程序运行后输出的二元式序列如下：

```
============================================================
  实验一  词法分析  ——  分析结果
  源程序：samples/test1.c
============================================================

【一】二元式序列（内部编码 ， 单词符号自身值）
------------------------------------------------------------
(2,"main")   (5,"(")     (5,")")     (5,"{")     (1,"int")   (2,"a")     (4,"=")
(3,"-5")     (5,",")     (2,"b")     (4,"=")     (3,"4")     (5,",")     (2,"j")
(5,";")      (1,"if")    (5,"(")     (2,"a")     (4,">=")    (2,"b")     (5,")")
(2,"j")      (4,"=")     (2,"a")     (4,"-")     (2,"b")     (5,";")     (1,"else")
(2,"j")      (4,"=")     (2,"b")     (4,"-")     (2,"a")     (5,";")     (5,"}")
```

测试数据一 运行结果

将上述输出与实验指导书给出的“要求输出”逐项对照，单词的类别编码、出现顺序及单词符号自身值完全一致，其中 -5 作为一个常数、>= 作为一个运算符、main 作为标识符处理。

（二）测试数据二

实验指导书给出的测试数据包含预处理指令、实型常数、字符串常数、地址运算符与逻辑运算符，源程序如下：

```
#include "math.h"

main()
{
  float a,b,y1,y2;
  int c,d,e;
  a=3.1415926;b=25.0;
```

```
    scanf("%d,%d",&c,&d);
    y1=cos(a/3);
    y2=sqrt(b);
    e=d+(-1);
    if(c>d&&c>0)  e=c;
    printf("y1=%f,y2=%f,e=%d,c=%x\n",y1,y2,e,c);
}
```

程序运行后输出的二元式序列如下（预处理指令行已跳过）：

```
============================================================
  实验一  词法分析  ——  分析结果
  源程序：samples/test2.c
============================================================

[预处理指令，已跳过] #include "math.h"

【一】二元式序列（内部编码 ，单词符号自身值）
------------------------------------------------------------
(2,"main")    (5,"(")      (5,")")      (5,"{")      (1,"float")    (2,"a")      (5,",")
(2,"b")       (5,",")      (2,"y1")     (5,",")      (2,"y2")       (5,";")      (1,"int")
(2,"c")       (5,",")      (2,"d")      (5,",")      (2,"e")        (5,";")      (2,"a")
(4,"=")       (3,"3.1415926")    (5,";")      (2,"b")      (4,"=")      (3,"25.0")    (5,";")
(2,"scanf")     (5,"(")      (3,"%d,%d")    (5,",")      (4,"&")      (2,"c")      (5,",")
(4,"&")       (2,"d")      (5,")")      (5,";")      (2,"y1")       (4,"=")      (2,"cos")
(5,"(")       (2,"a")      (4,"/")      (3,"3")      (5,")")        (5,";")      (2,"y2")
(4,"=")       (2,"sqrt")     (5,"(")      (2,"b")      (5,")")      (5,";")      (2,"e")
(4,"=")       (2,"d")      (4,"+")      (5,"(")      (3,"-1")       (5,")")      (5,";")
(1,"if")      (5,"(")      (2,"c")      (4,">")      (2,"d")      (4,"&&")     (2,"c")
(4,">")       (3,"0")      (5,")")      (2,"e")      (4,"=")      (2,"c")      (5,";")
(2,"printf")    (5,"(")      (3,"y1=%f,y2=%f,e=%d,c=%x\n")    (5,",")      (2,"y1")
(5,",")       (2,"y2")     (5,",")      (2,"e")      (5,",")      (2,"c")      (5,")")
(5,";")       (5,"}")
```

测试数据二 运行结果

（三）运行结果分析

两组测试数据运行时均未报告词法错误。对其中若干易错片段的识别结果及原因分析见表 3。

表 3　易错片段的识别结果分析

源程序片段	识别结果	说　明
#include "math.h"	整行跳过	预处理指令不属五大类单词，避免 math 被误判
float、int	(1,"float")、(1,"int")	命中保留字表，编码为 1
3.1415926、25.0	(3,"3.1415926")、(3,"25.0")	小数点并入常数，不单独切分
"%d,%d"	(3,"%d,%d")	字符串整体为一个常数，% 与 , 不拆开
d+(-1)	(3,"-1")	处于期待运算对象的位置，符号并入常数
c>d&&c>0	(4,">")、(4,"&&")、(4,">")	最长匹配,&& 不被拆成两个 &
a/3	(4,"/")	/ 后不是 / 或 *,按运算符处理而非注释
"…c=%x\n"	整个字符串	转义字符不截断字符串

对测试数据二的统计结果为：保留字 3 个，标识符 31 个，常数 7 个，运算符 13 个，分隔符 37 个，合计 91 个单词。其中 main、scanf、printf、cos、sqrt 等均按标识符处理，符合 C 语言的词法规则。

结　论

本次实验采用 C++ 语言，遵循“词法规则—状态转换图—自动机—扫描器”的设计流程，实现了一个针对 C 语言子集的词法分析器。程序通过 next() 函数输出五大类单词的二元式及行列号与统计信息，经测试验证，输出结果准确无误。

实验中的核心难点在于边界情况的精细化处理，包括通过上下文判断正负号归属、严格限制小数点后紧跟数字以区分实数与成员访问符，以及整行跳过预处理指令等。这些实践表明，严密的词法规则必须配合扎实的超前搜索，方能确保扫描器的鲁棒性。

通过本次实验，我深刻理解了最长匹配原则的核心思想，认识到自动机到达终态后仍需超前探测以确定单词边界。同时，将规则形式化为状态转换图使代码结构与路径完美对应，不仅便于排查错误，也为后续扩展关键字和运算符奠定了良好的可扩展性基础。